VINAY.T

GIT Interview Questions
DAY 3

Intermediate GIT Interview Questions

21.Can you explain the purpose and differences between the git diff and git status commands?

git status, What's going on in your repo?

✅ Purpose:
git status gives you a summary of the current state of your Git repository. It shows you which files have been:

- Modified (in working directory)
- Staged (in index, ready to commit)
- Untracked (new files not being tracked by Git)
- Deleted (either in working directory or staged)

🧠 How it works:
Git compares:

- Working Directory ⇔ Index (Staging Area)
- Index ⇔ HEAD (last commit)

Output Example:
$ git status
On branch main
Changes not staged for commit:
 (use "git add <file>..." to update what will be committed)
 modified: app.py

Untracked files:
 (use "git add <file>..." to include in what will be committed)
 newfile.txt

Changes to be committed:
 (use "git reset HEAD <file>..." to unstage)
 modified: readme.md

🧭 What this tells you:

- app.py has changes in the Working Directory, but is not staged.
- readme.md has changes that have been staged (ready to commit).
- newfile.txt is untracked — Git doesn't know about it yet.

git diff, What exactly changed?

✅ Purpose:
git diff shows the actual line-by-line changes in your files. You can use it to compare:

- Working Directory ⇔ Staging Area
- Staging Area ⇔ Last Commit (HEAD)
- Two commits
- Two branches

📌 Most Common Usages
1. git diff
Shows unstaged changes (Working Directory vs Index).
$ git diff

→ What did you change in the file that hasn't been staged yet?

2. git diff --cached or git diff --staged
Shows staged changes (Index vs HEAD).
$ git diff --cached

→ What will go into the next commit?

3. git diff HEAD
Shows all changes from HEAD (last commit) to the Working Directory, including both staged and unstaged.

🧠 Example to Understand:
Let's say you modify a file called hello.py.
$ echo "print('Hello World!')" >> hello.py

Now:
- This change is in Working Directory, not staged yet.

$ git diff
→ Shows what you just added (not staged yet).
$ git add hello.py
$ git diff --cached

→ Shows that this line is now staged for commit.
$ git status

→ Tells you that hello.py is staged and ready to commit.

🟦 Why is this important?
Understanding git diff and git status is crucial because:
- It helps avoid mistakes before committing.
- You get visibility into what's happening in your code.
- It clarifies what you're about to commit.
- It reinforces the three-tree architecture of Git (Working Dir, Index, HEAD).

22.How can you squash the last N commits into a single commit in Git? What command would you use, and what steps are involved?

🧠 What is Commit Squashing?
Squashing means combining multiple Git commits into a single commit. It's commonly used to:
- Clean up messy commit history before merging.
- Combine "work-in-progress" commits into a meaningful, atomic commit.
- Make code review easier by reducing noise in the history.

Why Squashing Overwrites History
When you squash commits using git rebase, you're rewriting commit history, changing commit hashes, order, and structure. This is safe if done in your local branch, before pushing to a shared repository. If done after pushing, it can cause divergence and conflict for others working on the same branch.

🔥 Always use caution: Never squash (or rebase) public commits unless everyone agrees and understands the implications.

🛠️ How to Squash Commits: git rebase -i HEAD~N
🔧 Command Syntax:
git rebase -i HEAD~N

- HEAD: Points to your current commit (usually the latest).
- HEAD~N: Refers to the last N commits.
- -i: Interactive mode — lets you choose how each commit should be handled.

📄 Step-by-Step Example
Let's say you made 4 commits like this:
git log --oneline
d5f4c3b Fix typo
a3e7b2d Add footer section
4c9a123 Update styles
1a2b3c4 Initial layout setup

You want to squash the last 4 commits into one.

1. Run the interactive rebase:
git rebase -i HEAD~4

2. You'll see an editor open with something like:
pick 1a2b3c4 Initial layout setup
pick 4c9a123 Update styles
pick a3e7b2d Add footer section
pick d5f4c3b Fix typo

3. Change all but the first pick to squash (or s):
pick 1a2b3c4 Initial layout setup
squash 4c9a123 Update styles
squash a3e7b2d Add footer section
squash d5f4c3b Fix typo

4. Save and close the editor.

5. Git then asks you to edit the new commit message. You can keep the existing ones or write a new, concise message:
Initial layout setup with footer and style updates

6. Save and close the editor again.
Now, your commit history will show one clean commit instead of 4.
📌 Result:
git log --oneline

e6d8f21 Initial layout setup with footer and style updates

✅ When to Use Squashing
- Before merging feature branches (e.g., feature/login) into main or develop.
- To clean up "fix typo", "WIP", or "test" commits.
- To prepare patches or PRs with clean history.

23.If a branch was pushed to the central repository but later accidentally deleted from all team members' local machines, how would you recover it?

🧠 Problem Context
You had a branch, let's call it feature-x; which was:
- Pushed to the central repository (like GitHub, GitLab, etc.).
- Accidentally deleted from all local machines.

Now, no one on the team has it locally, but it did exist remotely at some point.
The goal: Recover that deleted branch, ideally with all its commits.

✅ Solution: Use Git Reflog to Recover the Branch
Even if a branch is deleted, Git keeps track of where branches and commits used to be using something called the reflog.

🔍 What is reflog?
The reflog is a local history of all the changes made to HEAD, including branch checkouts, commits, rebases, merges, and deletions.

It can be used to find lost commits or deleted branches that were once referenced in your repo.

🛠️ Step-by-Step Guide to Recover the Deleted Branch
✅ 1. Run git reflog
git reflog

You'll see a list of recent actions, like:

d1e8b1c HEAD@{0}: checkout: moving from main to feature-x
a5c9d2a HEAD@{1}: commit: Add login validation
c3a4e9f HEAD@{2}: commit: Create login form
...

Look for the last commit that was part of the deleted branch. Let's say it was:
a5c9d2a HEAD@{1}: commit: Add login validation

✅ 2. Create a new branch from that commit
Now you can create a new branch from that commit:
git checkout -b feature-x a5c9d2a

This tells Git:
"Create a new branch called feature-x and point it to commit a5c9d2a."

✅ 3. Push it back to the remote (if needed)
If the remote branch is also gone and you want to restore it:
git push origin feature-x

🔐 Why Is Reflog So Powerful?

- It tracks all branch movements and commit references, even after deletion.
- It gives you a safety net in case of accidental branch deletion, bad rebase, or commit loss.
- Reflog entries are stored for 90 days by default, giving you a window for recovery.

📌 Note: Reflog is local. If every team member lost the branch and none of them have the relevant reflog entries, you'll need to recover from the central repository (if it still has the branch).

24.What is git reflog and how is it useful in Git? Can you provide examples of situations where you would use it?

🔍 What is git reflog?

The git reflog command is a powerful Git feature that tracks changes to references in your local repository, primarily HEAD, but also any branches or tags you've worked with.

In simpler terms, every time you move to a different commit, Git logs that movement, whether it's due to:

- Creating or switching branches
- Committing changes
- Merging
- Rebasing
- Resetting
- Pulling or fetching

Even if a commit or branch is deleted, git reflog remembers it — for a limited time — so you can recover lost work.

🧠 How Does Reflog Work?

Every Git reference (like HEAD, main, or feature-x) has its own reflog — a record of when and how that reference was updated.

✅ What it tracks:

- When a branch is created.
- When a branch is checked out.
- When a commit is made.
- When a merge, rebase, reset, or stash pop happens.
- When a branch is deleted, if it was local or previously checked out locally.

Each entry includes:

- The commit hash before the change.
- The commit hash after the change.
- The action (e.g., commit, checkout, merge).
- A timestamp.

📌 Important Details

🏷️ Reference tracking is local only:

- git reflog maintains logs only in your local repository.
- If a branch was never created or checked out on your local machine, its reference will not appear in your reflog.
- So, recovery is only possible if the branch or commit existed locally at some point.

📂 Applies to the current repository:
The git reflog command must be run in the same Git repository where the branch was lost or deleted. If the repository has no prior knowledge of the branch (i.e., it was never cloned, checked out, or committed to), there will be no reflog entry for it.

🛠️ Example: How Reflog Helps Recover Lost Work
Imagine this sequence:

```
git checkout -b feature-x
# make several commits
git checkout main
git branch -D feature-x  # accidentally deleted!
```

Oops! You deleted feature-x. But because it was created and worked on locally, Git stored its changes in the reflog.

You can recover it:
git reflog
Look for an entry like:
a5f9b2c HEAD@{3}: checkout: moving from feature-x to main

Then recover with:

```
git checkout -b feature-x a5f9b2c
```

You've now restored the lost branch using its last known commit reference from the reflog.

🧠 Key Takeaway
git reflog is like the black box flight recorder of your Git repo, it doesn't forget where you've been. Even if something is deleted or lost, as long as it existed locally, you can find your way back.

25.What information is stored inside a Git commit object? Can you explain its structure?

🧱 What Does a Git Commit Object Contain?
In Git, a commit object is a fundamental building block. It represents a snapshot of your project at a specific point in time, along with metadata that helps Git manage version history. Each commit object is uniquely identified and links your work into a chain of history.

🔹 Components of a Commit Object:

1. Snapshot of Tracked Files
A commit object stores a reference to a tree object, which records:

- The exact state of the project's files and directories at the time of the commit.
- File names, modes (permissions), and pointers to the content (stored as blobs in Git).

Unlike traditional systems that store differences (deltas), Git stores a complete snapshot of the project each time you commit (with optimizations like shared objects under the hood).

2. Parent Commit Reference(s)

- Most commits point to one parent commit, forming a linear history.
- Merge commits have multiple parents, representing the merge of multiple branches.
- The first commit (initial commit) has no parent.

This parent-child link forms a commit chain, allowing Git to reconstruct the complete history of changes.

3. Commit Hash (SHA-1 / Object ID)
Each commit object is uniquely identified by a 40-character SHA-1 hash (e.g., 9fceb02...). This hash is:

- Generated based on the contents of the commit (including tree, parents, author, message, etc.).
- Immutable — any change in commit contents results in a new hash.
- Used to track and refer to commits reliably.

4. Author and Committer Information

- Author: The person who originally wrote the code or change.
- Committer: The person who committed it (can be the same as the author).
- Includes names, email addresses, and timestamps.

This is useful for collaboration and auditing.

5. Commit Message
A human-readable description of the commit.

- Explains what and why the change was made.
- Critical for understanding project history and for effective collaboration.

26.Can you explain the different configuration levels in Git (git config) and how you can set values at each level?

🧠Understanding Git Configuration Levels
Git uses a hierarchical set of configuration files to determine how it behaves. These files are plain text files that define settings controlling everything from user identity to alias shortcuts, merge behavior, editor preferences, and more.

You can override Git's default behavior by modifying these configuration files. To do this effectively, it's important to understand how Git locates and prioritizes them.

🔺 Git Searches for Configuration in the Following Order:
1.System-Level Configuration

- Location:

Typically found at:
<<installation_path>>/etc/gitconfig

- Scope:

Applies to every user and every repository on the system.

- Usage:

To read or write to this file, use the --system flag:
git config --system user.name "Your Name"

2.Global/User-Level Configuration

- Location:

~/.gitconfig or ~/.config/git/config (depending on your system)

- Scope:

Applies to the current user across all repositories.

- Usage:

To modify global configurations, use the --global flag:
git config --global user.email "you@example.com"

Local (Repository-Level) Configuration

- Location:

.git/config file inside the repository's root directory

- Scope:

Applies only to the specific repository in which the command is executed.

- Usage:

To modify local settings (default scope if no flag is provided), use:
git config user.name "Repo Specific Name"

or explicitly with:
git config --local user.name "Repo Specific Name"

⚙️ Configuration Priority
If the same setting is defined at multiple levels, Git uses the following priority order (highest to lowest):

1. Local
2. Global
3. System

This means settings defined in the local repository will override user-level or system-wide settings.

📌 Final Note
If you run git config without specifying a level (e.g., --global, --local), Git defaults to the local repository-level configuration.

27.What is a detached HEAD state in Git, what typically causes it, and how can it be avoided or resolved?

🔍 What is a Detached HEAD in Git?
A detached HEAD state occurs when Git's HEAD pointer is not pointing to the latest commit of a named branch, but instead to a specific commit hash, tag, or any reference that isn't a local branch.

In this state, any new commits made are not attached to any branch, which means they can become "dangling" and risk being lost if not explicitly saved.

⚠ Common Scenarios That Lead to a Detached HEAD
1.Checking out a Tag or Commit Directly

- When you run:
  - git checkout <tag_name>
    # or
  - git checkout <commit_hash>

- Git moves HEAD to that specific point in history without switching to a branch.
- Any commit made from this point is not tied to a branch, making it a free-floating commit.

2.Working on a Read-Only Branch (e.g., remote tracking branch)

- If you try to commit directly while on a remote-only reference (e.g., origin/main), Git cannot link that commit to a local branch.
- As a result, the commit floats without a branch, leading to a detached HEAD.

🎯 Why Is This a Problem?

- Commits made in a detached HEAD state are not preserved unless explicitly saved.
- If you switch back to a branch without creating one first, Git will discard those commits during cleanup (garbage collection).
-

✅ How to Avoid or Resolve a Detached HEAD

To avoid entering a detached HEAD state or to safely work from a tag or commit:

1. Create a new branch from the commit or tag, and switch to it:
   git checkout -b <new-branch-name> <commit-hash or tag>

2. This ensures:

- Git attaches HEAD to the newly created branch.
- All commits from that point forward will be safely recorded under the new branch.
- You maintain a clear, connected project history.

📌 Pro Tip

If you accidentally commit in a detached HEAD state, you can still recover the commit using git reflog , as long as it's within the reflog retention window.

28.What is the purpose of the git annotate command, and how is it used in Git?

📄 What Does git annotate Do?

The git annotate command displays line-by-line information for a given file, showing which commit, author, and timestamp is responsible for each line in that file. It helps developers understand the history of changes at a very granular level.

It’s especially useful when you want to investigate:

- Who last modified a specific line.
- When and why a particular change was made.
- Which commit introduced a specific piece of code.

🧪 Syntax

git annotate [<options>] <file> [<revision>]

- <file> – the file to annotate.
- <revision> – optional. Specifies the commit or revision from which to start the annotation.

29.What is the difference between git stash apply and git stash pop, and when would you use each?

🔄 Difference Between git stash pop and git stash apply

1.git stash pop

- This command applies the most recent stash (or a specified one) and then immediately removes it from the stash list.
- It's a convenient way to reapply your changes and clean up the stash in one step.
- Example:
  git stash pop

2.git stash apply

- This command applies the changes from a stash but does not remove it from the stash list.
- Useful when you want to reapply the same stash multiple times or keep it for future use.
- To manually remove a stash after applying, use:
  git stash drop

  git stash pop = git stash apply + git stash drop